



ENGG100 Major Project Group P:314

Engineering Computing and Analysis (University of Wollongong)



Scan to open on Studocu

ENGG100

MAJOR

PROJECT

GROUP P/314

Oscar Tilden -	7481354
Alicia Bamforth -	7476528
Zane Beecher -	7446482
Georgia Caldwell -	7324601

Introduction

The aim of this report is to present a STRAVA bike ride into various plots. The plots created though had to work for two bike rides to identify that the code could be universally used. This was to be completed by utilising the bike rides longitude, latitude, time and elevation, which is derived from the rides GPX file. The plots produced from this data present these elements against each other. By utilising GPS data recorded by two students on two bike rides the following kinematic plots were required to be coded utilising MATLAB.

“A three-dimensional (3D) plot of the Easting (km) vs. Northing (km) vs. elevation (m) and additionally on the primary y-axis an elevation (m) vs. time graph, and on the secondary y-axis an elevation grade (%) vs. time graph. Furthermore, required to be plotted was a cumulative distance (km) vs. time graph, a velocity (km/h) vs. time graph and a velocity (km/h) vs. cumulative distance (km) graph.” (ENG100 Major Project Specification, 2022) Finally, there were also an additional two challenge problems to be completed using this data.

To create these plots and the code for them, various dynamics and programming concepts were required. This includes the use of both basic and advanced MATLAB coding. Further utilised in this project was the use of dynamics concepts which include rectilinear and curvilinear motion and calculus to find displacement, velocity and acceleration.

Background Theory

While creating a number of plots the background theory relating to dynamics had to be considered so that these problems could be solved. This firstly included the use of mathematical differentiation and integration in a number of plots.

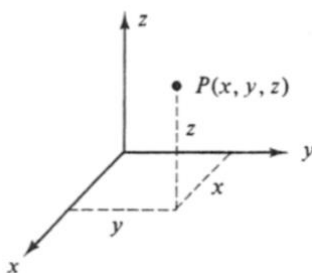
By differentiating displacement velocity can be found and then by differentiating this value acceleration can be found. The reverse order of this can be found further by integrating. This theory was utilised in the coding of plots. In terms of MATLAB coding these concepts of calculus were integrated into code utilising the *diff* function. This helped to produce plots that required the values of both velocity and acceleration which for the most part was essential throughout the project.

$$\begin{aligned} \frac{dx}{dt} &= v \\ \frac{dv}{dt} &= a \end{aligned}$$

Figure 1: differential values

As stated previously the aim of the project was to produce various kinematic plots relating to a bike ride. These plots are either rectilinear or curvilinear. The rectilinear plots relate to any function that lies in a straight line while curvilinear plots relate to those that sit on a curved line.

As these various kinematic graphs had to be plotted an understanding of the required coordinate system was needed. It is known that there are three specific coordinate systems: the cartesian coordinate system, the normal and tangential coordinate system and the cylindrical coordinate system.



The cartesian coordinate system (x,y,z) is a planar system made up of 2-3 perpendicular axis at the origin (0,0,0), It can be seen to the left.

The normal and tangential coordinate system (n, t) (Figure 3), which has its origin on the object, is a two-dimensional system for representing the motion of an item moving along a curved path. When the instant in question occurs, the t axis is perpendicular to the path and positive towards the direction of motion, while the n axis is perpendicular and positive towards the centre curve.

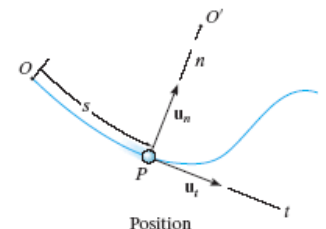
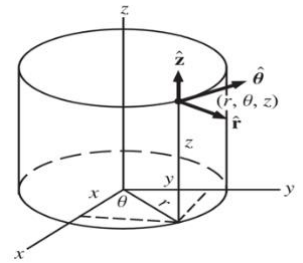


Figure 2: Cartesian Coordinate System

Figure 3: normal and tangential coordinate system

The final coordinate system considered is that of the cylindrical coordinate system (r, θ, z) (Figure 4) this is a 3D system that quantifies the location of a point with relation to a reference axis, direction, and reference plane that are all chosen. The reference plane and axis' intersection is where the origin is situated.

Figure 4: cylindrical coordinate system



The cartesian system was the sole suitable frame of reference from these several coordinate systems.

For the Easting and Northing coordinates and the Elevation, we had to use kilometres and metres, respectively, to produce the graph. Since the raw Easting, Northing, and elevation data were all expressed in metres and each point on the map represented a location on the globe, we had to subtract each point from the first number in each column to make sure the plot began at that location (0,0,0). To output an answer in kilometres, we have to divide the easting and northing values by 1000.

As it was required to produce a plot of cumulative distance vs time using points of elevation, eastings and northings for points on a now cartesian plain the following formula was utilised

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Furthermore, in regard to MATLAB coding low level I/O functions, vector notation, vector mathematics and vector tools found in the MATLAB vector toolbox were utilised to complete the project. I/O functions relate to input and output functions, they allow “the most direct control over reading or writing data to export text data files” (MathWorks Australia 2022). This included the use of *fopen* and *fread* commands to generate .txt files from .gpx files. This data was then utilised to produce essential data in the form of both eastings and northings. In addition to these functions basic MATLAB coding was also utilised throughout the project. This included the use of assigning variables in addition to the use of for loops. An additional importance was in the use of the plot function which enabled the creation of all required plots. Furthermore, utilised in the coding of the various required kinematic plots was the use of vector notation and required vector mathematics found in the MATLAB toolbox. Here this included the use of the colon operator “:” which can “can create **vectors**, subscript arrays, and specify for iterations.” (MathWorks Australia 2022)

For both challenge problem one and two additional theory was required to be utilised. Here for challenge problem one

$$\text{tangential acceleration} = \text{change in velocity} / \text{change in time}.$$

In addition, for challenge problem two various formulas were required to eventually derive the formula for power. This included the use of the resistance force due to gravity (F_g), rolling resistance force (F_r) and aerodynamic drag (F_a) formulas

$$F_g = g \times \sin(\arctan(\text{slope})) \times (M + m)$$

$$F_r = g \times \cos(\arctan(\text{slope})) \times (M + m) \times C_{rr}$$

$$F_a = 0.5 \times C_d \times A \times \rho \times (v + w)^2$$

Following the use of these formulas in challenge problem two subsequently power could be calculated utilising the following formula:

$$P = (F_g + F_r + F_a) \times v / (1 - \text{loss})$$

Overall Flowchart

Below is the flowchart for the entire project and its coding. This flowchart was then broken down further into individual flowcharts.

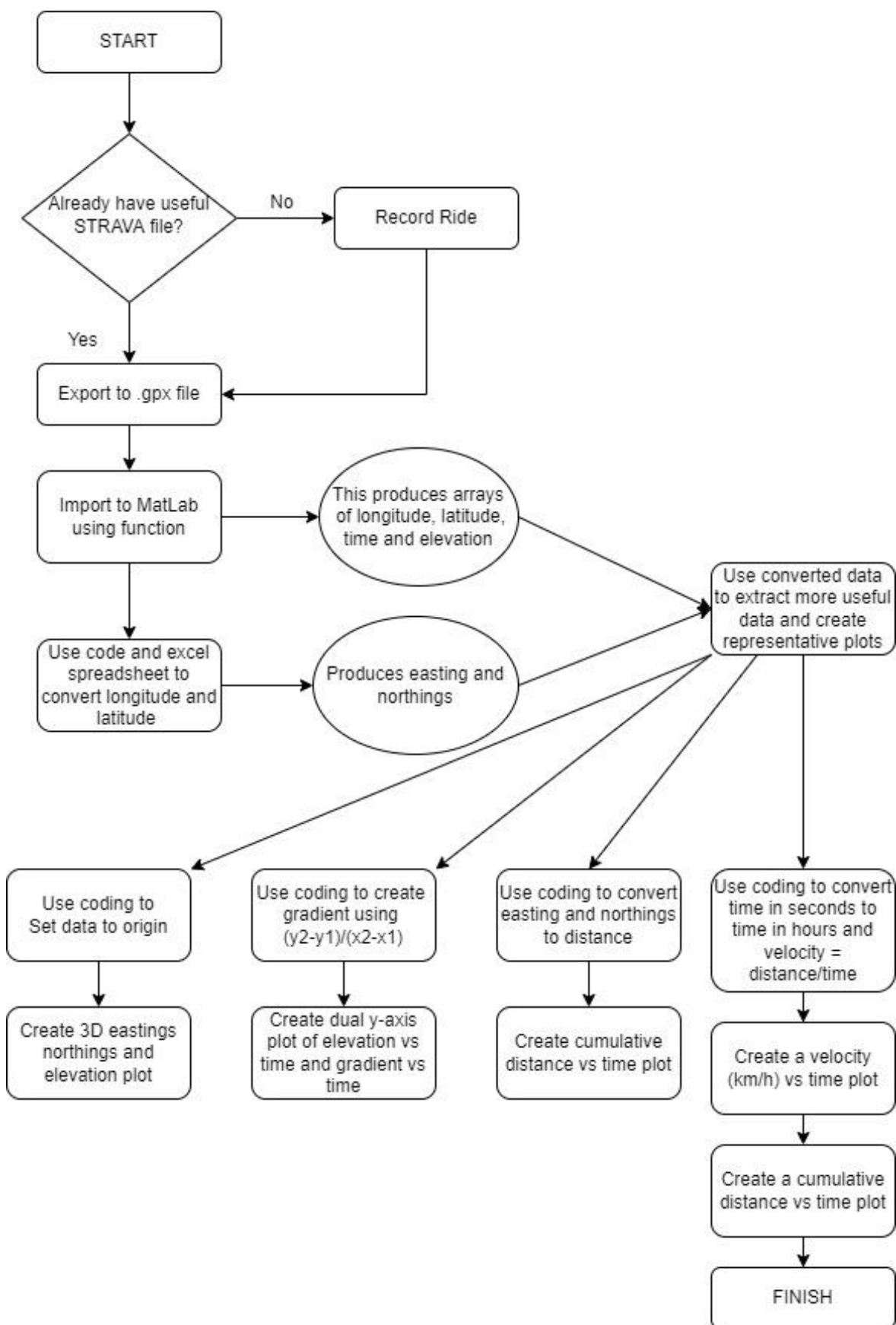


Figure 5: Overall Flowchart

Individual Flowcharts

Part 1,2 & 3 – Creation of data and data conversion.

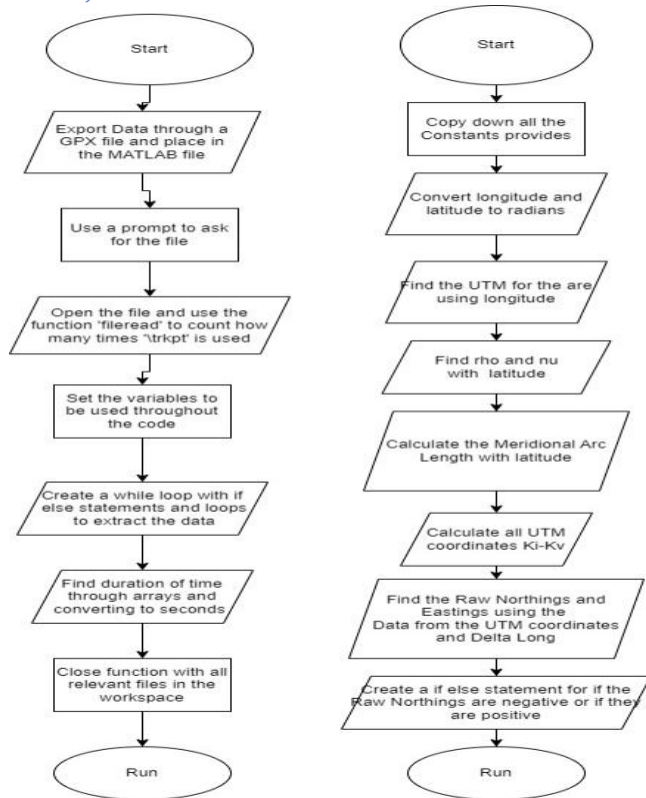


Figure 6: Part 2 Flowchart

Part 4a - Eastings, Northings and Elevation

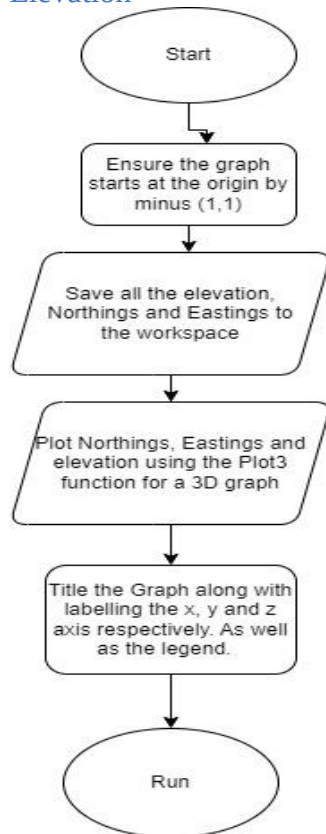


Figure 7: Part 4a Flowchart

Part 4b - Elevation and elevation grade vs time

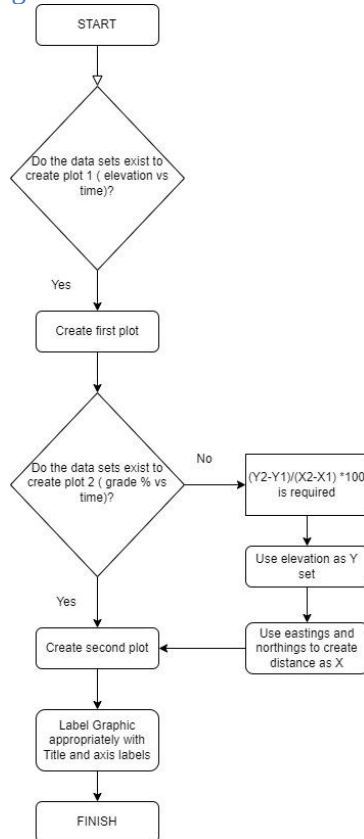


Figure 8: Part 4b Flowchart

Part 4c - Cumulative Distance vs Time

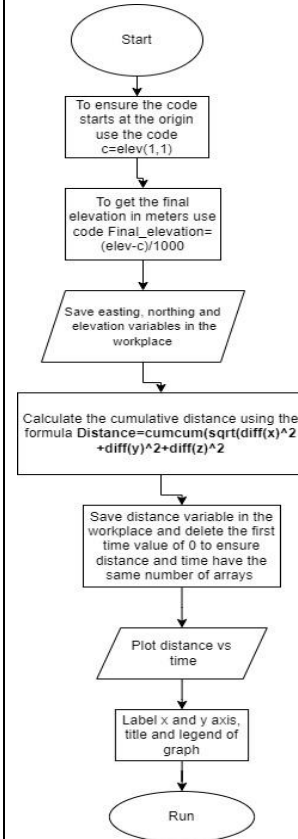


Figure 9: Part 4c Flowchart

Part 4d - Velocity vs Time

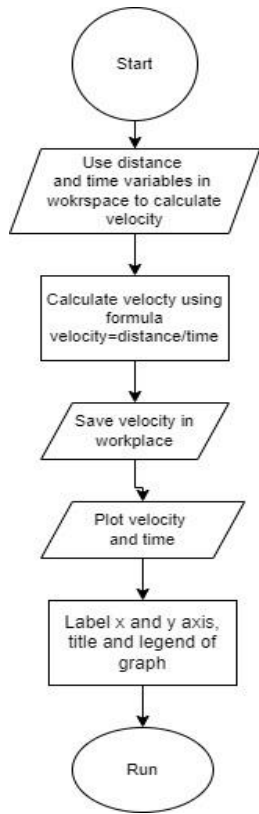


Figure 10: Part 4d Flowchart

Part 4e - Cumulative distance vs Velocity

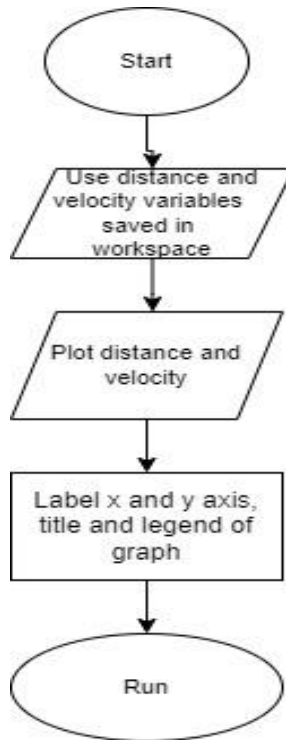


Figure 11: Part 4e Flowchart

Part 5 - Challenge Problem 1

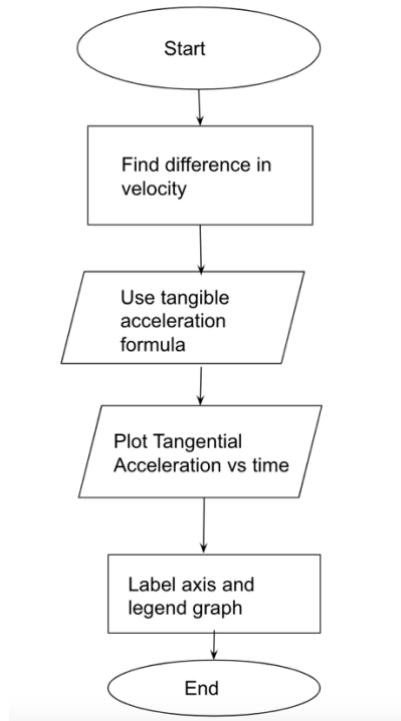


Figure 11: Part 5 Flowchart

Part 6 - Challenge Problem 2

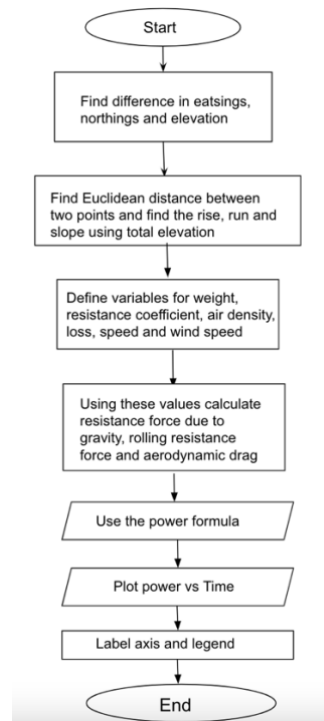


Figure 12: Part 6 Flowchart

The results of each flowchart saw the output of a plot which can be noted in the bellow sections in addition to further explanations about how these flowcharts operate to created these graphs.

Description of General Software

1. Part one of the project aimed to read and examine the raw data of the bike rides by utilising their GPX data files. These files were examined utilising low-level I/O (input/output) functions. These were utilised to read lines of code separately. By opening a data file, a while loop can then be utilised to scan each line of code until 'tempData' is read. From this point all points are put into a RawData matrix.
2. To utilise the data from the RawData matrix the points had to be converted to units. Here the data points in hours, minutes, and second in this matrix had to all be converted to seconds. Once completed the sum of seconds had to be found. This column created in the matrix of data can be utilised to find the difference in time. It finds the difference between each row in the matrix and creates an additional column for this now found change in time.
3. To generate the Cartesian coordinates from the points of longitude and latitude calculations involving assigned constants was conducted. These constants are applied at each point in the data set. Each assigned variable corresponded to one row in the data matrix. These constants were saved as variables and then calculations were performed on the first row. The Easting and Northing data were formulated as a result, and it was saved into a new RawData matrix column, up until it reached the final point of the column, this was repeated for each row.

4. Output Plots and Description of Software

a) Eastings(km), Northings(km) and Elevation (m)

With the elevation, easting, and northing data, a 3D graphic was intended to be produced. The Easting, Northing, and Elevation data from the data table is used to check if the first point is at the origin by deducting the first value from each subsequent value in the corresponding column. The Easting, Northing, and Elevation values were then plotted using the `plot3` command. The x and y axis were then converted to km by multiplying them by 1000.

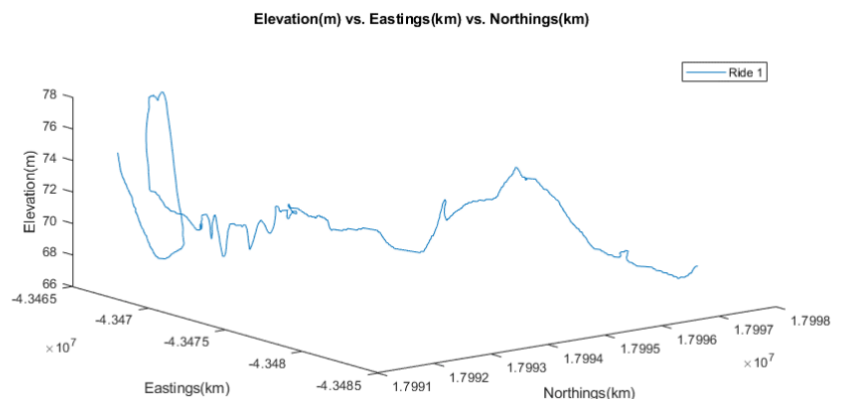


Figure 14: Plot 4a, Eastings, Northings and Elevation

b) Elevation and Elevation grade vs time

utilising raw height and time data in this section a plot of time and elevation was to be graphed. The plot function within MATLAB was used to graph the height of elevation in metres and time in seconds. This was formed into a 2D graph in order to illustrate the change in elevation per second. This was completed by using the raw data of elevation and the sum of all seconds from the previous section. The graph was further clarified with the addition of a title and two axis labels.

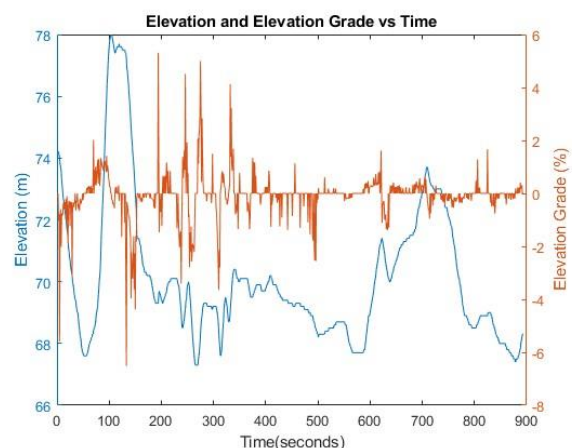


Figure 15: Plot 4b, elevation and elevation grade vs time

c) Cumulative distance vs time

In this section, a plot showing the cumulative sum of the distance versus time was to be created. To accomplish this, the plot had to begin at (0,0) and had the proper scale by using the difference in the eastings (x diff), northings (y diff), and elevation (z diff). This data was found in part 3. Using this data, the distance formula (as previously noted) was used to calculate the separation between each (x, y, z) point. The MATLAB command *cumsum* can then be used to find the distance's cumulative sum. These values had to be converted to km and thus were divided by 1000. This data was plotted against the time column from the data table.

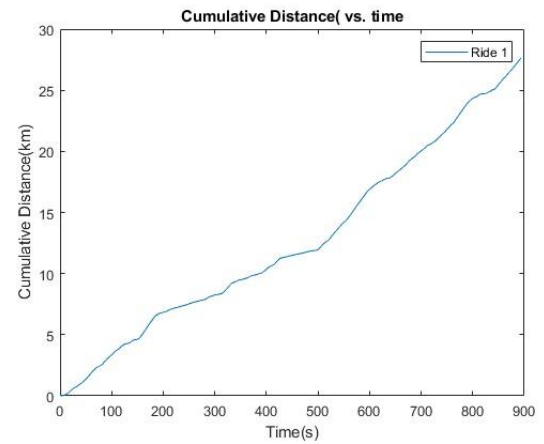


Figure 16: Plot 4c, Cumulative Distance vs Time

d) Velocity vs time

To produce a time vs velocity graph first the rate between the cumulative distance and time had to be computed. Alternatively the difference in distance could be obtained; and this value divided by the difference in time. Despite being slightly different from one another, these two approaches yield the same results. This discovered velocity is charted against time.

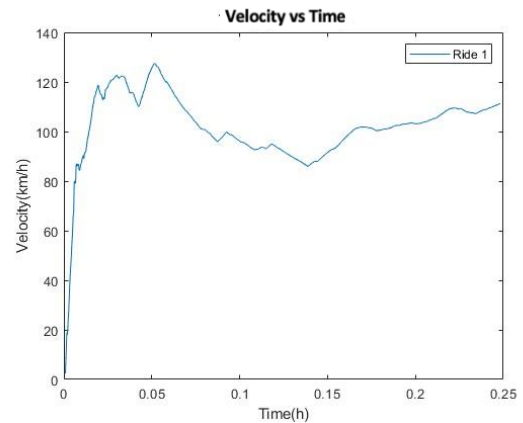


Figure 12: Plot 4d, Velocity vs Time

e) Velocity vs cumulative distance

The goal of this section was to create a graph of velocity vs cumulative distance. cumulative distance data was collected here by using the *cumsum* function. This distance data was found in unit metres using the following equation: “distance is the square root of the sum of the square roots of elevation, northings and eastings. This distance was converted to km. Velocity was then calculated by finding the change in distance and time. These values were divided to find the velocity in km per hour. The graph was then plotted

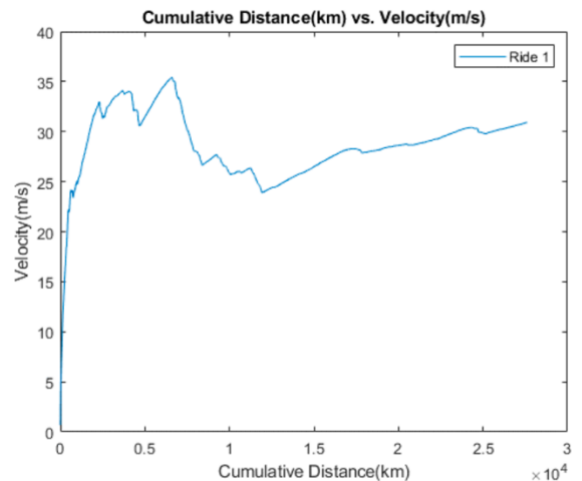


Figure 13: Plot 4e, Cumulative distance vs Velocity

Challenge Problems

a) Tangential acceleration vs time graph

Required by challenge problem one was to produce a tangential acceleration vs time graph. The difference in velocity can be calculated and utilised to find this value. Next, a 2D graph of the time and tangential acceleration was created using the plot function.

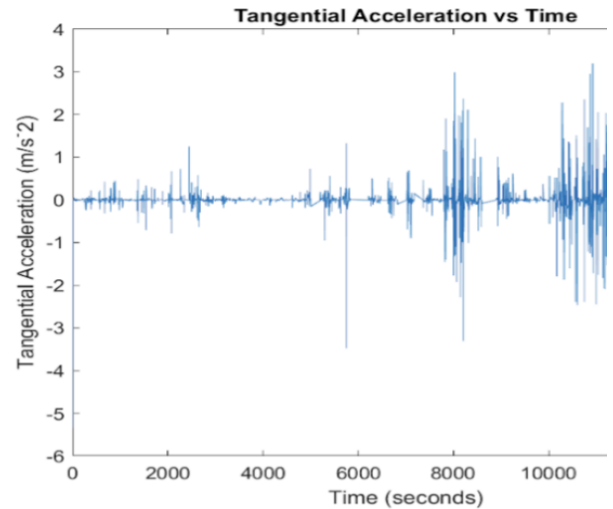


Figure 14: Tangential Acceleration vs Time

b) Cycling Power

Although the code found did not produce a plot, the aim of this challenge problem was to produce a cycling power vs time graph. This was to be achieved by using various formulas as outlined previously in the background theory section. The use of the resistance force due to gravity, rolling resistance force and aerodynamic drag formulas ultimately helped to produce the power formula in MATLAB. This produced a value for power, which was to be plotted against time. The codes that allowed to find values to help plot this function can be seen below.

```
g=9.81 %gravity constant
```

```
de=[1,diff(Easting)] %differentate easting values
dn=[1,diff(Northing_Final)]%differentate northing value
delev=[1,diff(elev)]%differentate elevation value
```

```
s=sqrt(dn.^2+de.^2+delev.^2) %Euclidean distance
between two points
```

```
for i= 1:length(s)
    distance = [1, sum(s(1:i))];
end
```

```
run = distance./1000; %formula to find run to find slop
```

```
for i=1:height(elev)
    totele=[1,sum(elev(1:i))] ;%total elevation
end
```

```
rise = totele./1000;
run= 5.38
```

```
slope = rise./run; %slope formula
```

```
M=80; %weight of rider
```

```
m=8; %weight of bike
```

```
Crr=0.025 %rolling resistance coefficnet (concrete)
```

```
cda=0.324 %drag coefficient
```

```
p=1.225 %air density above sea level
```

```
loss=0.04%loss
```

```
v=37.2%speed
```

```
w=12.4%wind speed
```

```
Fg= g.*sin(atan(slope)).*(M+m)
```

```
%resistance force due to gravity
```

```
Fr=g.*cos(atan(slope)).*(M+m).*Crr
```

```
%rolling resistance force
```

```
Fa=1/2.*cda.*p.*(v+w)
```

```
%aerodynamic drag
```

```
for i=1:length(Fg)
```

```

Fgg(i)=sum(Fg(1:i))
end

for i=1:length(Fr)
    Frr(i)=sum(Fr(1:i))
end

for i=1:length(Fa)
    Faa(i)=sum(Fa(1:i))
end

P=((Fgg+Frr+Faa).*v)./(1-loss)
%power formula

plot(P,time)
xlabel('Time (s)')
ylabel('Power(W)')
title ('Power(W) vs Time(s)')

```

Conclusion

To conclude, throughout the report we used both basic and advanced MATLAB skills, these skills were a crucial aspect to the report. We also used basic concepts of rectilinear and curvilinear motion and calculus to find displacement, velocity and acceleration. However, without extensive knowledge in MATLAB, our dynamics knowledge cannot be used as there would be no data to analysis. Our graphs are evidence that our report shows successful analysis of our STRAVA data. We know our graphs are correct because we constantly checked our values being calculated in MATLAB to ensure they are reasonable and realistic values. Furthermore, our report contains theoretical concepts to help prove our conclusion is valid and defensible. The cumulative distance is one of the main calculations used when producing the graphs, as it is used numerous times. We used the theoretical concept that $D = \text{cumsum}(\sqrt{\text{diff}(x)^2 + \text{diff}(y)^2 + \text{diff}(z)^2})$ in MATLAB to ensure this calculation was correct. Recommendations for future projects include, trying to keep the code as simple as possible to avoid any errors and create simplicity for people viewing or using the code and checking all calculations/variables being saved in the workspace to ensure the values are correct to avoid any future errors with calculations.

References

ENGG100 Major project Specification (2022) Moodle. [online] Available at: https://moodle.uowplatform.edu.au/pluginfile.php/3721773/mod_resource/content/2/ENGG100%20-%20Major%20Group%20Project%20Specification%202022%20V2.pdf (Accessed: October 24, 2022).

au.mathworks.com. (2022). *Matrices and Arrays - MATLAB & Simulink - MathWorks Australia*. [online] Available at: https://au.mathworks.com/help/matlab/learn_matlab/matrices-and-arrays.html.

Appendix – Full Code

<pre> %%Importing the gpx file %prompt input file name without'.gpx' file = input("Name of File: ", "s"); %strcat appends the.gpx to the file so the user doesn't have to file = strcat(file,".gpx"); %fopen, opens the file for binary read access filetext = fopen(file); %fileread, reads file as a character vector fulltext = fileread(file); %counts how many times '\trkpt' was used through the file %this will show how many lat and lon there are in the file lat = (count(fulltext,'\trkpt')); %set the variables to be used throughout the code the counter starting at 1 lat=lon; elev = lat; time = string(lat); counter = 1; %set the lines of the file to be ignored (IgnoreCase) and the lines in the %file that are useful to this assignment (extractBetween) %Use the function 'extractBetween' to take out these numbers in the text %between the given points and use 'str2double' to turn into the values and %place in a double array while ~feof(filetext) top = fgetl(filetext); if contains(top,'<trkpt','IgnoreCase',true)%if the line contains '<trkpt'the following equation it is 'true' and can be used lat(counter) = str2double(extractBetween(top,'lat=', ''));%inbetween'lat='and ' contain the latitude number which needs to be extracted lon(counter) = str2double(extractBetween(top,'lon=', ''));%inbetween'lon='and" contain the longitude number which needs to be extracted counter = counter + 1;%Create a loop within to extract all points wanted, this starts at point 1 else %if the line does no contain '<trkpt' then the function can end end%end function %repeat this if else statement with elevation and time if contains(top,'<ele>','IgnoreCase',true) elev(counter) = str2double(extractBetween(top,'<ele>','</ele>')); counter = counter + 1; else end if contains(top,'</time>','IgnoreCase',true) time(counter) = extractBetween(top,'T','Z'); counter = counter + 1; else end end %Find the duration by using an array starting at 2 to find the length %Convert the duration into seconds %Find the total duration by taking the final length and minusing by that %first length that was not included in the array time = time(2:length(time)); time = seconds(duration(time)); time = time-time(1); fclose all;%close the file %%Easting and Northings Calculations %Copy down all the Datum and Meridional Arc constants provided %Datum Constant a=6378137; b=6356752.3142; f=0.003352811; </pre>	<pre> %%Graphs % Set to the origin and change to km x=Easting(1,1) Final_Easting=(Easting-x)/1000 y=Northing(1,1) Final_Northings=(Northing-y)/1000 z=elev(1,1) Final_Elevation=(elev-z)/1000 %Graph a)Eastings vs. Northings vs. Elevation %Plot 3= plots three variable creating 3D plot (easting, northing, %elevation) plot3(Final_Easting,Final_Northing,Final _Elevation) %Label graph with x,y,z labels and legend title('Elevation(m) vs. Eastings(km) vs. Northings(km)') xlabel('Northings(km)') ylabel('Eastings(km)') zlabel('Elevation(m)') legend('Ride 1') %Graph b)Elevation vs. Time with Elevation grade(%) vs. Time %grade percentage is found by (rise/run) multiplied by 100. grade = rdivide(rise,run).*100; %plot elevation vs time graph on left y axis yyaxis left plot (time,elev) %plot elevation grade vs time graph on right y axis yyaxis right plot (time,grade) title ('Elevation and Elevation Grade vs Time') yyaxis left xlabel ('Time(seconds)') ylabel ('Elevation (m)') yyaxis right ylabel ('Elevation Grade (%)') %Graph c)Cumulative distance vs. Time %Access GPS.m to obtain useful data from the .gpx file %rise is the elevation data points from the .gpx file expressed as y2-y1 rise =elev(1,2:893) -elev(1,1:892); %to determine run longitude and latitude need to be converted into distance %distance is calculated by, sqrt= square root, diff is derivative of %subject distance=cumsum(sqrt(diff(Final_Northin g).^2 + diff(Final_Easting).^2 + diff(Final_Elevation).^2)); %run is the distance previously calculated expressed as x2-x1 run = distance(1,2:892)- distance(1,1:891); </pre>
---	---

<pre> rm=6367435.68; k0=0.9996; e=0.081819191; e1sq=0.006739497; n=0.00167922; %Meridional Arc Constants A0=6367449.146; B0=16038.42955; C0=16.83261333; D0=0.021984404; E0=0.000312705; %Calculation of inverse sin in pi sin_1=pi./(180.*3600); %Making sure the program understands that the lon and lat calculated when %importing the data is equal to the longitude and latitude longitude=(lon); latitude=(lat); %Calculating the UTM for the area %Use dots before multiplying, dividing and putting to a power of any vector %of the same length to make sure the function works %Use semi-colons so they don't carry through when evaluated Long_Zone=31+int(longitude./6); Long_Zone_CM=(6.*(Long_Zone)-183); Delta_Long=((longitude)-(Long_Zone_CM)).*3600./10000; %Converting longitude and latitude from degrees to radians Lat_Rad=(latitude).*pi./180; Long_Rad=(longitude).*pi./180; %Finding the rho and nu rho=a.*(1-e.*e)./((1-e.*sin(Lat_Rad).^2).^3./2); nu=a./(1-(e.*sin(Lat_Rad).^2).^1./2); %Calculate Meridional Arc Length which tells us how from the equator this %ride was done in latitude Meridional_Arc_S=A0.*Lat_Rad-B0.*sin(4.*Lat_Rad)- D0.*sin(6.*Lat_Rad)+E0.*sin(8.*Lat_Rad); %Coefficients for UTM Coordinates that will be used to find the Eastings %and Northings Ki=Meridional_Arc_S.*k0; Kii=nu.*sin(Lat_Rad).*cos(Lat_Rad).*(sin_1).^2.*k0.*(1000000000)./2; Kiii=((sin_1.^4.*nu.*sin(Lat_Rad).*cos(Lat_Rad).^3./24).*(5- tan(Lat_Rad).^2+9.*e1sq.*cos(Lat_Rad).^2+4.*e1sq.^2.*cos(Lat_Rad).^4).*k0.* (10000000000000000)); Kiv=nu.*cos(Lat_Rad).*sin_1.*k0.*10000; Kv=(sin_1.*cos(Lat_Rad)).^3.*(nu./6).*(1- tan(Lat_Rad).^2+e1sq.*cos(Lat_Rad).^2.*k0.*10000000000000); A6=((Delta_Long.*sin_1).^6.*nu.*sin(Lat_Rad).*cos(Lat_Rad).^5./720).*(61- 58.*tan(Lat_Rad).^2+tan(Lat_Rad).^4+270.*e1sq.*cos(Lat_Rad).^2- 330.*e1sq.*sin(Lat_Rad).^2).*k0.*(1E+24); %Calculate the final Easting and Raw Northings with the UTM coordinates and the Delta Long %variable Raw_Northing=(Ki+Kii.*Delta_Long.*Delta_Long+Kiii.*Delta_Long.^4); Easting=500000+(Kiv.*Delta_Long+Kv.*Delta_Long.*15.^3); %Create a if else statement for if the Raw_Northings found were negative or %positive %Must finish with an end to signify that the function must stop if Raw_Northing<0 Northing=1000000+Raw_Northing else Northing=Raw_Northing end %use the function clearvars so only the arrays of Easting, Northings, time, %file and elev which are all needed for the graphing clearvars -except Easting Northing time file elev </pre>	<pre> %cumsum is the function used to calculate the cumulative sum in our case we calculated the cumulative distance %function plot() plots the graph of our variables plot(time,distance) %label all aspects of graph title('Cumulative Distance(km) vs. time(s)') xlabel('Time(s)') ylabel('Cumulative Distance(km)') legend('Ride 1') %d)Velocity vs. Time %Convert time in seconds to hours to calculate velocity in km/h %Use calculated distance from above/ variable of time to get velocity hours=time./3600; velocity=distance./hours; %function plot() plots the graph of our variables plot(hours,velocity) %label all aspects of graph title('Time(h) vs. Velocity') xlabel('Time(h)') ylabel('Velocity(km/h)') legend('Ride 1') %e)Velocity vs. Cumulative distance with given variables already calculated plot(distance,velocity) %label all aspects of graph title('Cumulative Distance(km) vs. Velocity(km/h)') xlabel('Cumulative distance(km)') ylabel('Velocity(km/h)') legend('Ride 1') %% CHALLENGE 1 tangential acceleration (m/s^2) vs. time graph velocity_ms = velocity_kmhr./3.6 ; %convert to m/s vDiff = [0; diff(velocity_ms)]; %difference in velocity at = velocity_ms.*(vDiff./RawData(:, 17)); %tangential acceleration formula Figure= plot(time2, at) title('Tangential Acceleration vs Time') xlabel('Time (seconds)') ylabel('Tangential Acceleration (m/s^-2)') % Challenge Problem 2 g=9.81 %gravity constant de=[1,diff(Easting)] %differentiate easting values dn=[1,diff(Northing_Final)]%dif ferentiate northing value </pre>
---	---

```

delev=[1,diff(elev)]%differentiate elevation value
s=sqrt(dn.^2+de.^2+delev.^2)
%Euclidean distance between two points
for i= 1:length(s)
    distance = [1,
sum(s(1:i))];
end
run = distance./1000; %formula to find run to find slope
for i=1:height(elev)
    totele=[1,sum(elev(1:i))]
;%total elevation
end
rise = totele./1000;
run= 5.38
slope = rise./run; %slope formula
M=80; %weight of rider
m=8; %weight of bike
Crr=0.025 %rolling resistance coefficient (concrete)
cda=0.324 %drag coefficient
p=1.225 %air density above sea level
loss=0.04%loss
v=37.2%speed
w=12.4%wind speed
Fg= g.*sin(atan(slope)).*(M+m)
%resistance force due to gravity
Fr=g.*cos(atan(slope)).*(M+m).*Crr
%rolling resistance force
Fa=1/2.*cda.*p.*(v+w)
%aerodynamic drag
for i=1:length(Fg)
    Fgg(i)=sum(Fg(1:i))
end
for i=1:length(Fr)
    Frr(i)=sum(Fr(1:i))
end
for i=1:length(Fa)
    Faa(i)=sum(Fa(1:i))
end
P=((Fgg+Frr+Faa).*v)./(1-loss)
%power formula
plot(P,time)
xlabel('Time (s)')
ylabel('Power(W)')
title ('Power(W) vs Time(s)')

```